Lab 5 Static Code Analysis Report

Software Engineering Lab | UE23CS341A | PES University

14 October 2026

Objective

Improve the museum artifact catalog's code quality, security, and style using Pylint, Bandit, and Flake8. The required deliverables are a cleaned Python program with at least four issues fixed, a completed issue table, and answers to the four reflection questions.

Program and analysis method

The assigned museum_catalog.py uses an in-memory SQLite database. It registers artifacts, reports their display status, moves them onto display, and totals their value by era. Its backup function prints a notice; it does not perform a backup.

The original program was inspected and executed before running the three analyzers. The findings were documented, the medium severity SQL issue was prioritized, and the code was corrected. All tools were rerun against the cleaned program, followed by behavior checks.

Environment: Python 3.12.10 in a virtual environment; Pylint 4.1.3, Bandit 1.9.4, and Flake8 7.4.1.

Verification results

Check	Before fixes	After fixes
Pylint	17 messages; score 4.85/10	No messages; score 10.00/10
Bandit	5 findings: 1 medium, 4 low	No issues identified
Flake8	8 messages	No findings; exit code 0
Program	Successful demo; exit code 0	Successful demo; exit code 0
Regression tests	Not run on original source	8 tests passed

All final analyzers returned exit code 0. No analyzer rules were disabled and no security findings were suppressed. The final demonstration prints:

```
on display
135000
catalog backup complete
```

The original run printed the backup notice first because it used a child process. The cleaned program prints the same three messages in program order and retains the Chola total of 135000.

Evidence: the original and final reports are retained in evidence/baseline and evidence/final. The required root report files are pylint_report.txt, bandit_report.txt, and flake8_report.txt.

Identified issues and fixes

The table uses line numbers from the original assigned file, preserved at commit 7c1fc2f. Bandit found one medium and four low severity issues; there were no high severity findings. The medium finding was fixed first.

Issue	Type	Line(s)	Description	Fix Approach
B608 SQL construction	Security Medium	11	Artifact fields were concatenated into the INSERT statement, allowing input to affect SQL syntax.	Used four ? placeholders and a separate value tuple. Quoted titles and SQL-like input are stored literally.
B105 Hardcoded password	Security Low	7	An unused curator password was stored directly in source.	Removed the unused credential. This removes exposed dead code; it does not add authentication.
B602 Shell execution	Security Low	37	A fixed echo command used shell=True. Bandit classified this constant command as low severity.	Replaced the subprocess call with Python print to retain the notice without a shell.
B607 Partial path	Security Low	37	The same subprocess call used an executable name without an absolute path.	Removed the unnecessary process invocation.
B404 Subprocess import	Security review Low	3	The subprocess import was flagged for security review. The import alone does not prove exploitation.	Removed the import after replacing the shell call.

Additional robustness improvements

Manual inspection also found missing-ID handling and field validation problems. Empty text, negative or unsupported values, and invalid display flags are now rejected. Unknown IDs raise clear ValueError messages. The connection is closed after use, and importing the module does not run the demonstration. These improvements were not baseline analyzer findings.

Code quality and style fixes

The following rows complete the required issue table. Pylint's quality categories and Flake8's style codes are separate from Bandit's security severity levels. Overlapping messages can describe the same defect.

Issue	Type	Line(s)	Description	Fix Approach
W0611 / F401 Unused import	Quality	2	datetime was imported but never used.	Removed the unused import.
W0613 Unused argument	Quality	15	The curator argument did not affect displayStatus.	Removed the unused argument and updated the demonstration call.
R1705 Redundant else	Refactor	17-20	The if branch returned, making the following else unnecessary.	Used a direct conditional return after checking whether the artifact exists.
C0103 Function names	Convention	10, 15, 24, 31, 36	Five function names used camelCase instead of snake_case.	Renamed the catalog functions using snake_case and updated all call sites.
C0114 / C0116 Documentation	Convention	1; 10, 15, 24, 31, 36, 40	The module and six functions lacked docstrings.	Added module and function docstrings describing their purpose and relevant errors.
C0301 / E501 Long lines	Style	6, 11, 16, 26, 32	SQL and calls exceeded the tools' default line-length limits.	Split fixed SQL strings and calls across parenthesized lines below 80 characters.
E303 Blank lines	Style	24	Three blank lines preceded totalValue.	Used two blank lines between top-level functions.
E225 Assignment spacing	Style	25	total=0 lacked spaces around the assignment operator.	Applied consistent spacing when rewriting the value calculation.

All reported catalog findings were resolved, exceeding the minimum of four fixes. The final Pylint score also exceeds the repository's target of 9.5. The final Flake8 report is empty because there are no findings; its recorded exit code confirms successful execution.

Reflection

1. What is static code analysis, and why is it useful?

Static code analysis examines source code without executing the program. It helps identify possible programming errors, security weaknesses, and style problems early in development. In this lab, the museum catalog ran successfully before any changes, but analysis still found unsafe SQL construction, a hardcoded password, unused code, and formatting problems. Finding these issues early makes the code easier to maintain and reduces the chance that defects reach users.

2. What is the difference between Pylint, Bandit, and Flake8?

Pylint checks Python code quality, including naming conventions, missing docstrings, unused imports and arguments, and unnecessary branching. Bandit looks for security-sensitive patterns, such as constructing SQL from strings, hardcoded passwords, and shell execution. Flake8 checks style and common code errors, including long lines, whitespace, excess blank lines, and unused imports. Their checks complement one another: Pylint and Flake8 both found the unused datetime import, while Bandit identified the SQL security concern.

3. What issues were identified and how were they addressed?

Bandit reported five findings: concatenated SQL, an unused hardcoded curator password, the subprocess import, a partial executable path, and shell=True. The medium severity SQL finding was fixed first by using four question-mark placeholders and a separate tuple of values. This keeps artifact titles and eras as data and supports apostrophes. The unused password was removed. The subprocess call only printed a fixed notice, so it was replaced with Python print, removing the shell and subprocess import.

Pylint and Flake8 also reported missing docstrings, camelCase function names, an unused datetime import and curator argument, a redundant else, long lines, extra blank lines, and missing assignment spacing. These were fixed by adding documentation, using snake_case names, removing unused code, simplifying the return, and formatting the file. Input validation and clear handling of unknown artifact IDs were also added after manual inspection. Rerunning the tools produced Pylint 10.00/10 and no Bandit or Flake8 findings.

4. How does static code analysis improve software quality?

Static analysis improves readability, consistency, and maintainability by finding issues before execution. Security checks draw attention to risky patterns, while naming, documentation, and formatting checks make the code easier to understand and review. These checks support testing and code review; they do not replace them. The baseline tools did not flag every robustness problem, such as unknown artifact IDs. Eight regression tests therefore checked the demonstration, display transitions, collection totals, quoted and SQL-like input, missing IDs, invalid fields, and safe module import. All passed, and the Chola collection total remained 135000.